Cuaderno de Entrenamiento Didáctico: LISP

Programación Funcional — De la Inseguridad al Dominio Práctico

Cátedra de Paradigmas y Lenguajes de Programación Preparación Intensiva para Exámenes y Recuperatorios

Alumna: ..... **Fecha de inicio:** ..... **Docente / Ayudante:** Santiago Scetti

📌 Guía de Uso del Cuaderno

Este cuaderno está organizado en **bloques progresivos**. No tenés que hacer todo de golpe: cada módulo desarma una de las dificultades típicas que aparecen en los parciales.

Regla de oro: Intentá resolver primero en lápiz, forzándote a "ser la computadora". Recién después verificamos en la terminal de Lisp.

MÓDULO 1: Anatomía de Listas y Cirugía de Accesores

💡 Conceptos Clave

- `(car lista)` : extrae el **primer elemento** (átomo o sublista).
- `(cdr lista)` : extrae el **resto de la lista** (siempre es una lista, o `nil` al vaciarse).
- **Regla de dos pasos para sublistas:**
  Paso 1: Moverse hasta la sublista con `cdr` sucesivos y entrar con `car` .
  Paso 2: Extraer el campo dentro de esa sublista (`car` para el 1º, `cadr` para el 2º, `caddr` para el 3º).
- **Constructores:** `cons` agrega un elemento al inicio; `append` concatena dos listas; `list` envuelve sus argumentos en una nueva lista.

Ejercicio 1.1 — Verdadero o Falso: Estructura de Listas

[ ] V [ ] F **1.** La expresión `(cdr '(42))` devuelve `nil`.

Justificación:

[ ] V [ ] F **2.** Si evaluamos `(cons '(1 2) '(3 4))`, obtenemos la lista plana `(1 2 3 4)`.

Justificación:

[ ] V [ ] F **3.** En la lista `((A B) (C D) (E F))`, la llamada `(caadr ...)` extrae el elemento `B`.

Justificación:

[ ] V [ ] F **4.** La función `(append '(A B) '(C D))` da el mismo resultado que `(list '(A B) '(C D))`.

Justificación:

[ ] V [ ] F **5.** En LISP, tanto `(car nil)` como `(cdr nil)` devuelven `nil` sin causar error.

Justificación:

Ejercicio 1.2 — Predicción de Retornos: ¿Qué devuelve LISP?

Dada la variable `M = '(CAFE (BRASIL MEDIO) 1500 (LECHE AZUCAR))`, indicá el retorno exacto:

Expresión	Retorno de LISP (con paréntesis si es lista)
`(car M)`	.....
`(cdr M)`	.....
`(cadr M)`	.....
`(caddr M)`	.....
`(car (cadr M))`	.....
`(cadr (cadr M))`	.....
`(cadddr M)`	.....
`(car (cadddr M))`	.....

Ejercicio 1.3 — Completar Accesores Quirúrgicos

Dado el registro de pedido: `P = '(4 (JUAN TARDE) 3 4500 PAGADO)` con estructura: `(NUM-MESA (MOZO TURNO) CANT-ITEMS TOTAL-PESOS ESTADO)`

```
;; a) Obtener el número de mesa (ej: 4):
(defun obtener-mesa (p)
    (_________________ p))

;; b) Obtener el nombre del mozo (ej: JUAN):
(defun obtener-mozo (p)
    (car (_________________ p)))

;; c) Obtener el turno del mozo (ej: TARDE):
(defun obtener-turno (p)
    (cadr (_________________ p)))

;; d) Obtener el total a pagar (ej: 4500):
(defun obtener-total (p)
    (cadddr p))  ;; ¿Qué combinación equivalente con car y cdr?: (car (cdr (cdr (_____
```

```
;; e) Obtener el estado del pedido (ej: PAGADO):
(defun obtener-estado (p)
    (car (cddddr p)))  ;; o mediante nth: (nth ______ p)
```

Ejercicio 1.4 — Caza de Bichos: El Error de Susana

Una alumna debía obtener los nombres de los mozos de una lista de clientes y escribió esto:

```
(defun nombres-mozos (lista)
    (list (car (cdr (car lista)))
          (car (cdr (car (cdr lista))))
          (car (cdr (car (cdr (cdr lista)))))))
```

1. ¿Funciona si la lista trae 5 mesas o solo 2 mesas? ¿Por qué?

2. ¿Qué concepto de programación funcional falta entender al resolver posición por posición a mano?

MÓDULO 2: Lógica Condicional y Predicados Puros

💡 Conceptos Clave

- **Predicados puros:** Devuelven exclusivamente `T` o `nil`. Por convención terminan en `-p`.
- **Comparación de datos:** Usá `=`, `<`, `>` solo para *números*. Para símbolos y listas usá siempre `equal`.
- **IF vs COND (Regla de examen):**
  Usa `if` únicamente con 2 ramas exactas (true / false).
  Usa `cond` cuando hay 3 o más alternativas.
  PROHIBIDO en la cátedra: Anidar un `if` dentro de un `cond` o viceversa.
- **Estructura de COND:** Cada cláusula es `((condicion) resultado)`. Si falta el resultado, el cond devuelve el valor de la condición (usualmente `T`).

Ejercicio 2.1 — Verdadero o Falso: Decisiones y Tipos

[ ] V [ ] F **1.** La comparación `(= 'PAGADO 'PAGADO)` evalúa a `T` sin errores.

Justificación:

[ ] V [ ] F **2.** Si en `(cond ((< x 0) -1) ((> x 0) 1))` pasamos `x = 0`, LISP arroja un error en consola.

Justificación:

[ ] V [ ] F **3.** La rama `(T valor)` al final de un `cond` funciona como el caso por defecto (else).

Justificación:

[ ] V [ ] F **4.** Si una función predicado devuelve el número `0`, LISP lo interpreta como falso (`nil`).

Justificación:

[ ] V [ ] F **5.** La expresión `(and (numberp 25) (> 25 10) 'DESCUENTO)` devuelve `T`.

Justificación:

Ejercicio 2.2 — Predicción de Retornos: Clasificador de Cafés

```
(defun clasificar-pedido (precio minutos)
    (cond
        ((not (and (numberp precio) (numberp minutos))) 'DATOS-INVALIDOS)
        ((and (≥ precio 3000) (≤ minutos 5))          'PREMIUM-EXPRESS)
        ((≥ precio 3000)                              'PREMIUM)
        ((≤ minutos 5)                                'EXPRESS)
        (T                                            'ESTANDAR)))
```

Llamada de evaluación	Retorno de la función
`(clasificar-pedido 3500 4)`	.....
`(clasificar-pedido 3200 12)`	.....
`(clasificar-pedido 1500 3)`	.....
`(clasificar-pedido 2000 10)`	.....
`(clasificar-pedido 'MUCHO 5)`	.....

Ejercicio 2.3 — Completar Código: Categorizar Consumo

Categorizar según monto: < 2000: `'BASICO`, de 2000 a 5000: `'REGULAR`, > 5000: `'VIP`.

```
(defun categorizar-consumo (monto)
    (cond
        ((not (numberp monto)) _____________________)
        ((< monto 2000)        'BASICO)
        ((≤ monto 5000)        _____________________)
        (T                     'VIP)))
```

Ejercicio 2.4 — Caza de Bichos: El Error de Valen

```
(defun listo-para-entregar-p (estado minutos)
    (cond
        ((equal estado 'LISTO))
        ((and (equal estado 'PREPARANDO) (> minutos 15)) 'URGENTE)
        (T 'ESPERAR)))
```

1. ¿Qué devuelve si evaluamos `(listo-para-entregar-p 'LISTO 5)`?

2. ¿Por qué ocurre esto y cómo se arregla la primera línea del cond?

MÓDULO 3: El Molde Sagrado de la Recursividad

💡 La Tabla Sagrada de Neutros y Combinadores

Patrón	Caso Base (null l)	Operación en paso recursivo
Contar cuántos cumplen	`0`	`(+ 1 (fun (cdr l)))`
Sumar un campo o total	`0`	`(+ (campo (car l)) (fun (cdr l)))`
Filtrar (nueva lista de sublistas)	`nil`	`(cons (car l) (fun (cdr l)))`
Universal (¿Todos cumplen?)	`T`	Corta con `nil` al primer fallo; si no, avanza.
Existencial (¿Al menos uno?)	`nil`	Corta con `T` al primer acierto; si no, avanza.

Regla de oro: El caso base `((null lista) neutro)` va **siempre primero** en el `cond`. Y toda llamada recursiva debe achicar la lista con `(cdr lista)`.

Ejercicio 3.1 — Verdadero o Falso: Mecánica Recursiva

[ ] V [ ] F 1. En una función que cuenta elementos, el caso base `((null lista) nil)` es correcto.

Justificación:

[ ] V [ ] F 2. Escribir `(sumar lista)` en vez de `(sumar (cdr lista))` provoca desbordamiento de pila (stack overflow).

Justificación:

[ ] V [ ] F 3. Para filtrar elementos se debe usar `+` en la rama verdadera para encadenar las sublistas.

Justificación:

[ ] V [ ] F 4. El caso base debe ir primero para evitar evaluar `(car nil)` antes de tiempo.

Justificación:

Ejercicio 3.2 — Trazado Manual: La Pila de Llamadas

Dada la función `sumar-precios` y la lista `'(1000 2000 1500)`:

```
(defun sumar-precios (lista)
    (cond
        ((null lista) 0)
        (T (+ (car lista) (sumar-precios (cdr lista))))))
```

Llamada	(car lista)	¿Caso base?	Cálculo pendiente al desapilar
`(sumar-precios '(1000 2000 1500))`	1000	NO	`(+ 1000 ...)`
`(sumar-precios '(2000 1500))`	.....	NO	`(+ 2000 ...)`
`(sumar-precios '(1500))`	.....	NO	`(+ 1500 ...)`
`(sumar-precios '())`	nil	**SÍ**	Devuelve neutro: .....
Resultado final tras resolver todas las sumas pendientes:			**Rta:** .....

Ejercicio 3.3 — Completar los Tres Patrones de Examen

```
;; PATRÓN A: Contar cuántos superan un precio límite
(defun contar-caros (lista limite)
    (cond
        ((null lista) ________)  ;; ← ¿Qué neutro va al contar?
        ((> (car lista) limite)
            (+ 1 (contar-caros (____________ lista) limite)))
        (T
            (contar-caros (cdr lista) limite))))

;; PATRÓN B: Filtrar pedidos para llevar (armar lista con cons)
(defun filtrar-para-llevar (pedidos)
    (cond
        ((null pedidos) ________)  ;; ← ¿Qué neutro va al armar lista?
        ((equal (caddr (car pedidos)) 'LLEVAR)
            (cons (____________ pedidos)
                  (filtrar-para-llevar (cdr pedidos))))
        (T
            (filtrar-para-llevar (____________ pedidos)))))

;; PATRÓN C: Predicado Universal (¿Todas las mesas libres?)
(defun todas-libres-p (mesas)
    (cond
        ((null mesas) ________)  ;; ¿Si no quedó ninguna ocupada es T o NIL?
        ((not (equal (car mesas) 'LIBRE)) ________)  ;; Al primer fallo: corta
        (T (todas-libres-p (cdr mesas)))))
```

MÓDULO 4: MAPCAR y Funciones de Orden Superior

💡 **Cuándo usar MAPCAR vs cuándo usar Recursión**

- **MAPCAR:** Siempre que la lista de salida deba tener **exactamente la misma longitud** que la de entrada (transformación elemento a elemento, 1 a 1).
- **RECURSIÓN:** Siempre que haya que **filtrar** (la salida tiene menos elementos), **acumular** (suma o total), **contar**, o buscar con **corte temprano**.
- **Cierre Léxico:** Todo parámetro escalar (como un porcentaje o límite) debe ser capturado directamente por el `lambda`, **nunca** pasado como argumento extra a `mapcar`.

Ejercicio 4.1 — ¿Recursión o MAPCAR? (Toma de Decisiones)

Para cada consigna, marcá la herramienta adecuada y justificá según el largo de la lista:

1. *"Dada una lista de pedidos `((ID TIPO PRECIO) ...)`, obtener la lista de precios de cada pedido."*
[ ] MAPCAR [ ] RECURSIÓN

Justificación:

2. *"Dada una lista de pedidos, obtener una lista conteniendo únicamente los pedidos con precio > $4000."*
[ ] MAPCAR [ ] RECURSIÓN

Justificación:

3. *"Dada una lista de consumos, determinar el monto promedio consumido por mesa."*
[ ] MAPCAR [ ] RECURSIÓN

Justificación:

4. *"Dada una lista de tiempos de espera, transformar cada tiempo a un mensaje ('RAPIDO', 'NORMAL', 'DEMORADO')."*
[ ] MAPCAR [ ] RECURSIÓN

Justificación:

5. *"Dadas dos listas de números de igual longitud, obtener una nueva lista sumando posición a posición."*
[ ] MAPCAR [ ] RECURSIÓN

Justificación:

6. *"Determinar si existe al menos un pedido cuyo estado sea 'URGENTE'."*
[ ] MAPCAR [ ] RECURSIÓN

Justificación:

Ejercicio 4.2 — Predicción de Retornos con MAPCAR

Expresión MAPCAR	Retorno de LISP
`(mapcar 'cadr '((1 CAFE 2000) (2 TE 1500) (3 SUB 2800)))`	.....
`(mapcar '+ '(1000 2000 3000) '(200 300 400))`	.....
`(mapcar (lambda (x) (> x 1500)) '(1000 1500 2000 3000))`	.....
`(mapcar (lambda (p) (if (≥ p 2500) 'CARO 'ACCESIBLE)) '(2000 3500 1800 2500))`	.....
`(mapcar (lambda (cant prec) (* cant prec)) '(2 3 1) '(1500 2000 4000))`	.....

Ejercicio 4.3 — Completar Código: Lambda y COND con MAPCAR

Convertir cada pedido `'(ID PRECIO MINUTOS)` en una etiqueta de cocina:

```
(defun evaluar-tiempos-cocina (lista-pedidos)
    (mapcar (lambda (pedido)
                (cond
                    ((> (caddr pedido) 15)  _____________________)
                    (( ≥ (caddr pedido) 6)  'EN-HORA)
                    (T                      _____________________)))
            _____________________))
```

Ejercicio 4.4 — Caza de Bichos: El Parámetro Escalar

```
(defun aplicar-descuento-todos (lista porcentaje)
    (mapcar (lambda (precio desc) (* precio (- 1 desc)))
            lista
            porcentaje))
```

1. ¿Qué error arroja al llamar `(aplicar-descuento-todos '(2000 3000) 0.10)`?

---

2. Escribí la versión corregida capturando `porcentaje` mediante cierre léxico:

```
(defun aplicar-descuento-todos (lista porcentaje)
    (mapcar (lambda (precio)
                (* precio (- 1 ________________)))
            ________________))
```

MÓDULO 5: El Gimnasio Didáctico (Pareo y Depuración)

Ejercicio 5.1 — Pareo de Enunciados a Plantillas LISP

Asociá cada consigna en lenguaje cotidiano con la plantilla de código Lisp correspondiente:

Consigna	Plantilla LISP
1. [ ] Contar cuántos elementos cumplen un criterio. 2. [ ] Generar una lista con un mensaje para cada uno. 3. [ ] Obtener solo los elementos que cumplen un filtro. 4. [ ] Verificar si absolutamente todos cumplen. 5. [ ] Calcular la suma acumulada de un campo.	**A)** `(cond ((null l) T) ((not (p (car l))) nil) (T (f (cdr l))))` **B)** `(cond ((null l) 0) (T (+ (campo (car l)) (f (cdr l)))))` **C)** `(mapcar (lambda (x) (cond ((c1 x) 'M1) (T 'M2))) lista)` **D)** `(cond ((null l) nil) ((p (car l)) (cons (car l) (f (cdr l)))) (T (f (cdr l))))` **E)** `(cond ((null l) 0) ((p (car l)) (+ 1 (f (cdr l)))) (T (f (cdr l))))`

Ejercicio 5.2 — Caza de 4 Bichos Históricos de Exámenes

Bicho 1 (COND sin retorno):

```
(defun estado-cafetera (temperatura)
    (cond
        ((and (numberp temperatura) (> temperatura 95)))
        (T 'NORMAL)))
```

¿Qué devuelve `(estado-cafetera 100)` y cómo se corrige?

Bicho 2 (Comparador numérico en símbolos):

```
(defun es-descafeinado-p (tipo-cafe)
    (= tipo-cafe 'DESCAFEINADO))
```

¿Por qué explota y qué función debe usarse en su lugar?

Bicho 3 (Uso de variable global mutable):

```
(defun sumar-items (lista)
    (cond
        ((null lista) total)
        (T (setq total (+ total (car lista))) (sumar-items (cdr lista)))))
```

Escribilo de forma funcional pura (sin `setq`):

```
(defun sumar-items (lista)
    (cond
        ((null lista) ____)
        (T (+ _________________ (sumar-items _________________)))))
```

MÓDULO 6: Gran Simulacro Integrador — Cafetería "El Buen Grano"

📋 Estructura de Datos del Examen

Cada pedido individual tiene la forma:

```
(ID_PEDIDO (TIPO_GRANO TOSTADO) TAMANO PRECIO_BASE MINUTOS_ESPERA)
```

Ejemplo: `'(101 (COLOMBIA MEDIO) GRANDE 3500 4)`

Parte A: Operaciones sobre un Pedido Individual

```
;; 1. Funciones selectoras modulares
(defun obtener-grano (pedido)
    _____________________________________________)

(defun obtener-tostado (pedido)
    _____________________________________________)

(defun obtener-precio (pedido)
    _____________________________________________)

(defun obtener-tiempo (pedido)
    _____________________________________________)

;; 2. Predicado de grano colombiano (devuelve T o NIL)
(defun es-colombia-p (pedido)
    ____________________________________________________________________)

;; 3. Clasificador de tiempo de espera con COND: '(ID ETIQUETA)
;;  ⩽ 5: 'EXPRESS, de 6 a 12: 'NORMAL, > 12: 'DEMORADO
(defun clasificar-espera (pedido)
    (cond
        ((⩽ (obtener-tiempo pedido) 5)  (list (car pedido) 'EXPRESS))
        ((⩽ (obtener-tiempo pedido) 12) (list (car pedido) _________________))
        (T                              (list (car pedido) _________________))))
```

Parte B: Operaciones sobre la Lista de Pedidos

Lista de ejemplo para pensar:

```
((101 (COLOMBIA MEDIO) GRANDE 3500 4) (102 (BRASIL FUERTE) CHICO 2800 14) (103 (ETIOPI
```

4. Clasificación masiva con MAPCAR:

```
(defun informe-tiempos (lista-pedidos)
    (________________ 'clasificar-espera lista-pedidos))
```

5. Recaudación total acumulada (RECURSIÓN pura):

```
(defun recaudacion-total (lista-pedidos)
    (cond
        ((null lista-pedidos) ______)
        (T (+ (obtener-precio (car lista-pedidos))
              (recaudacion-total (cdr lista-pedidos))))))
```

6. Filtrar pedidos de Colombia (RECURSIÓN y CONS):

```
(defun pedidos-colombianos (lista-pedidos)
    (cond
        ((null lista-pedidos) ______)
        ((es-colombia-p (car lista-pedidos))
            (cons (car lista-pedidos)
                  (pedidos-colombianos (cdr lista-pedidos))))
        (T
            (pedidos-colombianos (cdr lista-pedidos)))))
```

7. Contar pedidos demorados (RECURSIÓN):

```
(defun contar-demorados (lista-pedidos tiempo-limite)
    (cond
        ((null lista-pedidos) 0)
        ((> (obtener-tiempo (car lista-pedidos)) tiempo-limite)
            (+ 1 (contar-demorados (cdr lista-pedidos) tiempo-limite)))
        (T
            (contar-demorados (cdr lista-pedidos) tiempo-limite))))
```

8. Predicado universal: ¿Todos fueron Express (<= 5 min)? (RECURSIÓN):

```
(defun todos-express-p (lista-pedidos)
    (cond
        ((null lista-pedidos) ______)
        ((> (obtener-tiempo (car lista-pedidos)) 5) ______)  ;; Corte temprano
        (T (todos-express-p (cdr lista-pedidos)))))
```

9. Aplicar descuento con MAPCAR y Lambda:

```
(defun aplicar-promo (lista-pedidos porcentaje)
    (mapcar (lambda (pedido)
                (* (obtener-precio pedido) (- 1 porcentaje)))
            lista-pedidos))
```

Parte C: Función Menú Principal (Interacción y Validación)

```
(defun menu-cafeteria ()
    (let ((pedidos) (tiempo-max))
        (print "═══ SISTEMA DE GESTION CAFETERIA EL BUEN GRANO ═══")
        (print "Ingrese la lista de pedidos: ")
        (setq pedidos (read))
        (print "Ingrese el tiempo maximo de espera aceptable (minutos): ")
        (setq tiempo-max (read))

        (if (and (consp pedidos) (numberp tiempo-max))
            (progn
                (print "──────────────────────────────────────────────────────")
                (print "INFORME DE TIEMPOS DE ESPERA:")
                (print (informe-tiempos pedidos))

                (print "RECAUDACION TOTAL DEL DIA ($):")
                (print (recaudacion-total pedidos))

                (print "PEDIDOS DE CAFE COLOMBIANO:")
                (print (pedidos-colombianos pedidos))

                (print "CANTIDAD DE PEDIDOS DEMORADOS:")
                (print (contar-demorados pedidos tiempo-max))

                (print "TODOS LOS PEDIDOS FUERON EXPRESS? (T/NIL):")
                (print (todos-express-p pedidos))

                (print "PRECIOS CON 10% DE DESCUENTO:")
                (print (aplicar-promo pedidos 0.10))
                (print "──────────────────────────────────────────────────────"))
            (print "ERROR: Los datos ingresados no son validos. Revise la lista y el n
```

🏁 Checklist de Seguridad antes de Entregar

- [ ] ¿El caso base está siempre **primero** en el `cond` ?
- [ ] ¿Usaste `0` para contar/sumar y `nil` para armar listas con `cons` ?
- [ ] ¿Todas las llamadas recursivas avanzan con `(cdr lista)` ?
- [ ] ¿No hay ninguna variable global (`setq` fuera de `let`)?
- [ ] ¿No anidaste `if` dentro de `cond` ni viceversa?
- [ ] ¿Comparaste los símbolos con `equal` en vez de con `=` ?
- [ ] ¿El nombre en la llamada recursiva coincide exactamente letra por letra con el `defun` ?